

Chương 3 – Phân loại (Classification)

 Open in Colab

 Open in Kaggle

```
# Thiết lập (Setup)
```

Dự án này yêu cầu Python 3.7 trở lên:

```
import sys

assert sys.version_info >= (3, 7)
```

Nó cũng yêu cầu Scikit-Learn $\geq 1.0.1$:

```
from packaging import version
import sklearn

assert version.parse(sklearn.__version__) >= version.parse("1.0.1")
```

Giống như trong chương trước, hãy xác định kích thước phông chữ mặc định để làm cho

```
import matplotlib.pyplot as plt

plt.rc('font', size=14)
plt.rc('axes', labelsz=14, titlesz=14)
plt.rc('legend', fontsize=14)
plt.rc('xtick', labelsz=10)
plt.rc('ytick', labelsz=10)
```

Và hãy tạo thư mục `images/classification` (nếu nó chưa tồn tại) và định nghĩa hàm

```
from pathlib import Path
```

```

IMAGES_PATH = Path() / "images" / "classification"
IMAGES_PATH.mkdir(parents=True, exist_ok=True)

def save_fig(fig_id, tight_layout=True, fig_extension="png", resolution=300):
    path = IMAGES_PATH / f"{fig_id}.{fig_extension}"
    if tight_layout:
        plt.tight_layout()
    plt.savefig(path, format=fig_extension, dpi=resolution)

```

```
# Bộ dữ liệu MNIST
```

```

from sklearn.datasets import fetch_openml

mnist = fetch_openml('mnist_784', as_frame=False)

```

```

# extra code - it's a bit too long
print(mnist.DESCR)

```

```

**Author**: Yann LeCun, Corinna Cortes, Christopher J.C. Burges
**Source**: [MNIST Website](http://yann.lecun.com/exdb/mnist/) - Date unknown
**Please cite**:

```

The MNIST database of handwritten digits with 784 features, raw data available as a NumPy array. It is a subset of a larger set available from NIST. The digits have been size-normalized and centered in a subset of the 10x10 pixel image. With some classification methods (particularly template-based methods, such as SVM), the MNIST dataset can be used to benchmark performance. The MNIST training set is composed of 30,000 patterns from SD-3 and 30,000 patterns from SD-1. Downloaded from openml.org.

```
mnist.keys() # extra code - we only use data and target in this notebook
```

```
dict_keys(['data', 'target', 'frame', 'categories', 'feature_names',
'target_names', 'DESCR', 'details', 'url'])
```

```

X, y = mnist.data, mnist.target
X

```

```

array([[0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       ...,
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.]])

```

X.shape

(70000, 784)

y

array(['5', '0', '4', ..., '4', '5', '6'], dtype=object)

y.shape

(70000,)

28 * 28

784

```
import matplotlib.pyplot as plt

def plot_digit(image_data):
    image = image_data.reshape(28, 28)
    plt.imshow(image, cmap="binary")
    plt.axis("off")

some_digit = X[0]
plot_digit(some_digit)
save_fig("some_digit_plot") # extra code
plt.show()
```



y[0]

'5'

```
# extra code - this cell generates and saves Figure 3-2
plt.figure(figsize=(9, 9))
for idx, image_data in enumerate(X[:100]):
    plt.subplot(10, 10, idx + 1)
    plot_digit(image_data)
plt.subplots_adjust(wspace=0, hspace=0)
save_fig("more_digits_plot", tight_layout=False)
plt.show()
```



```
X_train, X_test, y_train, y_test = X[:60000], X[60000:], y[:60000], y[60000:]
```

```
# Huấn luyện một bộ phân loại nhị phân (Binary Classifier)
```

```
y_train_5 = (y_train == '5') # True for all 5s, False for all other digits
y_test_5 = (y_test == '5')
```

```
from sklearn.linear_model import SGDClassifier
```

```
sgd_clf = SGDClassifier(random_state=42)
sgd_clf.fit(X_train, y_train_5)
```

```
SGDClassifier(random_state=42)
```

```
sgd_clf.predict([some_digit])
```

```
array([ True])
```

Đánh giá hiệu năng

Đo lường độ chính xác bằng kiểm định chéo (Cross-Validation)

```
from sklearn.model_selection import cross_val_score

cross_val_score(sgd_clf, X_train, y_train_5, cv=3, scoring="accuracy")

array([0.95035, 0.96035, 0.9604 ])
```

```
from sklearn.model_selection import StratifiedKFold
from sklearn.base import clone

skfolds = StratifiedKFold(n_splits=3) # add shuffle=True if the dataset is not
                                     # already shuffled
for train_index, test_index in skfolds.split(X_train, y_train_5):
    clone_clf = clone(sgd_clf)
    X_train_folds = X_train[train_index]
    y_train_folds = y_train_5[train_index]
    X_test_fold = X_train[test_index]
    y_test_fold = y_train_5[test_index]

    clone_clf.fit(X_train_folds, y_train_folds)
    y_pred = clone_clf.predict(X_test_fold)
    n_correct = sum(y_pred == y_test_fold)
    print(n_correct / len(y_pred))
```

```
0.95035
0.96035
0.9604
```

```
from sklearn.dummy import DummyClassifier

dummy_clf = DummyClassifier()
dummy_clf.fit(X_train, y_train_5)
print(any(dummy_clf.predict(X_train)))
```

False

```
cross_val_score(dummy_clf, X_train, y_train_5, cv=3, scoring="accuracy")
```

```
array([0.90965, 0.90965, 0.90965])
```

Ma trận nhầm lẫn (Confusion Matrix)

```
from sklearn.model_selection import cross_val_predict
```

```
y_train_pred = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3)
```

```
from sklearn.metrics import confusion_matrix
```

```
cm = confusion_matrix(y_train_5, y_train_pred)
cm
```

```
array([[53892,   687],
       [ 1891,  3530]])
```

```
y_train_perfect_predictions = y_train_5 # pretend we reached perfection
confusion_matrix(y_train_5, y_train_perfect_predictions)
```

```
array([[54579,    0],
       [    0,  5421]])
```

Độ chính xác (Precision) và Độ triệu hồi (Recall)

```
from sklearn.metrics import precision_score, recall_score
```

```
precision_score(y_train_5, y_train_pred) # == 3530 / (687 + 3530)
```

```
0.8370879772350012
```

```
# extra code - this cell also computes the precision: TP / (FP + TP)
cm[1, 1] / (cm[0, 1] + cm[1, 1])
```

```
0.8370879772350012
```

```
recall_score(y_train_5, y_train_pred) # == 3530 / (1891 + 3530)
```

```
0.6511713705958311
```



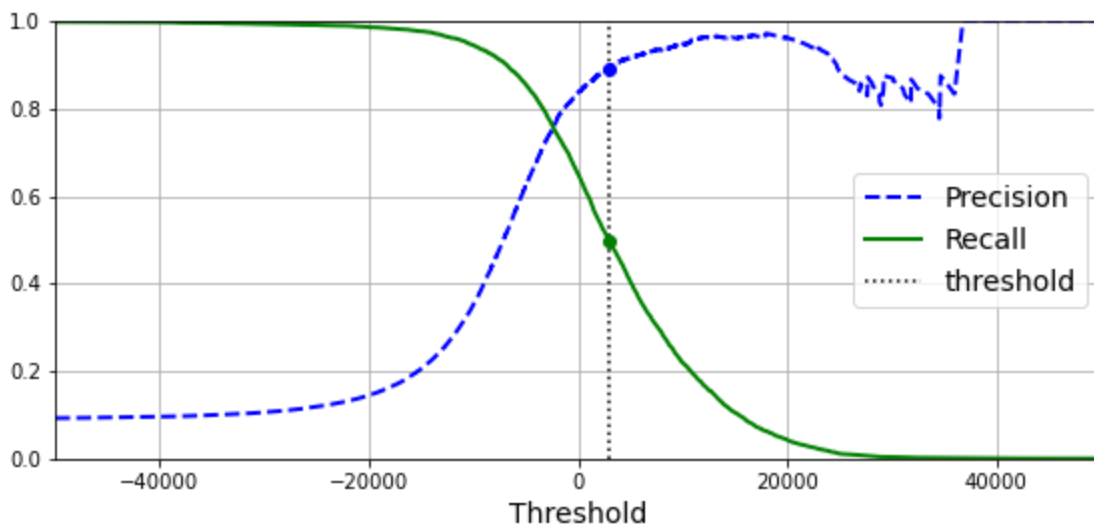
```
from sklearn.metrics import precision_recall_curve
```

```
precisions, recalls, thresholds = precision_recall_curve(y_train_5, y_scores)
```

```
plt.figure(figsize=(8, 4)) # extra code - it's not needed, just formatting
plt.plot(thresholds, precisions[:-1], "b--", label="Precision", linewidth=2)
plt.plot(thresholds, recalls[:-1], "g-", label="Recall", linewidth=2)
plt.vlines(threshold, 0, 1.0, "k", "dotted", label="threshold")
```

```
# extra code - this section just beautifies and saves Figure 3-5
idx = (thresholds >= threshold).argmax() # first index  $\geq$  threshold
plt.plot(thresholds[idx], precisions[idx], "bo")
plt.plot(thresholds[idx], recalls[idx], "go")
plt.axis([-50000, 50000, 0, 1])
plt.grid()
plt.xlabel("Threshold")
plt.legend(loc="center right")
save_fig("precision_recall_vs_threshold_plot")
```

```
plt.show()
```



```
import matplotlib.patches as patches # extra code - for the curved arrow
```

```
plt.figure(figsize=(6, 5)) # extra code - not needed, just formatting
```

```
plt.plot(recalls, precisions, linewidth=2, label="Precision/Recall curve")
```

```
# extra code - just beautifies and saves Figure 3-6
```

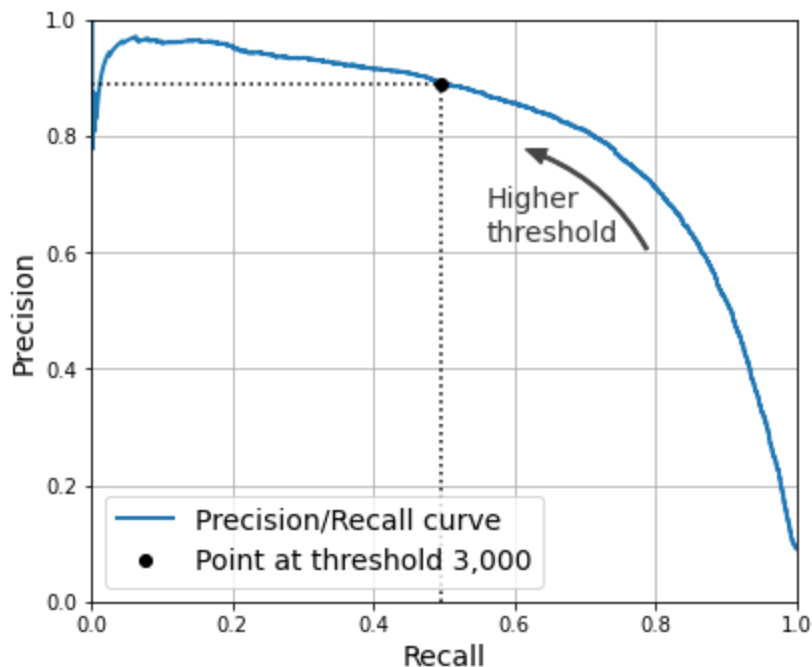
```
plt.plot([recalls[idx], recalls[idx]], [0., precisions[idx]], "k:")
plt.plot([0.0, recalls[idx]], [precisions[idx], precisions[idx]], "k:")
plt.plot([recalls[idx], [precisions[idx]], "ko",
         label="Point at threshold 3,000")
plt.gca().add_patch(patches.FancyArrowPatch(
    (0.79, 0.60), (0.61, 0.78),
    connectionstyle="arc3,rad=.2",
```

```

        arrowstyle="Simple, tail_width=1.5, head_width=8, head_length=10",
        color="#444444"))
plt.text(0.56, 0.62, "Higher\ nthreshold", color="#333333")
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.axis([0, 1, 0, 1])
plt.grid()
plt.legend(loc="lower left")
save_fig("precision_vs_recall_plot")

plt.show()

```



```

idx_for_90_precision = (precisions >= 0.90).argmax()
threshold_for_90_precision = thresholds[idx_for_90_precision]
threshold_for_90_precision

```

```
3370.0194991439557
```

```
y_train_pred_90 = (y_scores >= threshold_for_90_precision)
```

```
precision_score(y_train_5, y_train_pred_90)
```

```
0.9000345901072293
```

```

recall_at_90_precision = recall_score(y_train_5, y_train_pred_90)
recall_at_90_precision

```

```
0.4799852425751706
```

Đường cong ROC

```
from sklearn.metrics import roc_curve
```

```
fpr, tpr, thresholds = roc_curve(y_train_5, y_scores)
```

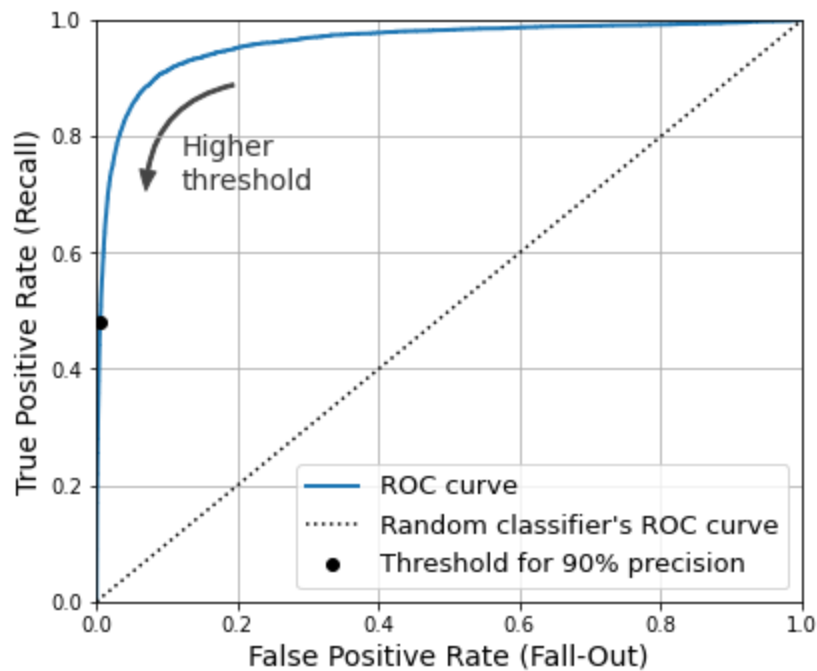
```
idx_for_threshold_at_90 = (thresholds <= threshold_for_90_precision).argmax()  
tpr_90, fpr_90 = tpr[idx_for_threshold_at_90], fpr[idx_for_threshold_at_90]
```

```
plt.figure(figsize=(6, 5)) # extra code - not needed, just formatting  
plt.plot(fpr, tpr, linewidth=2, label="ROC curve")  
plt.plot([0, 1], [0, 1], 'k:', label="Random classifier's ROC curve")  
plt.plot([fpr_90], [tpr_90], "ko", label="Threshold for 90% precision")
```

```
# extra code - just beautifies and saves Figure 3-7  
plt.gca().add_patch(patches.FancyArrowPatch(  
    (0.20, 0.89), (0.07, 0.70),  
    connectionstyle="arc3,rad=.4",  
    arrowstyle="Simple, tail_width=1.5, head_width=8, head_length=10",  
    color="#444444"))
```

```
plt.text(0.12, 0.71, "Higher\ntreshold", color="#333333")  
plt.xlabel('False Positive Rate (Fall-Out)')  
plt.ylabel('True Positive Rate (Recall)')  
plt.grid()  
plt.axis([0, 1, 0, 1])  
plt.legend(loc="lower right", fontsize=13)  
save_fig("roc_curve_plot")
```

```
plt.show()
```



```
from sklearn.metrics import roc_auc_score
```

```
roc_auc_score(y_train_5, y_scores)
```

```
0.9604938554008616
```

Warning: the following cell may take a few minutes to run.

```
from sklearn.ensemble import RandomForestClassifier
```

```
forest_clf = RandomForestClassifier(random_state=42)
```

```
y_probas_forest = cross_val_predict(forest_clf, X_train, y_train_5, cv=3,
                                     method="predict_proba")
```

```
y_probas_forest[:2]
```

```
array([[0.11, 0.89],
       [0.99, 0.01]])
```

These are *estimated probabilities*. Among the images that the model classified as positive with a probability between 50% and 60%, there are actually about 94% positive images:

```
# Not in the code
```

```
idx_50_to_60 = (y_probas_forest[:, 1] > 0.50) & (y_probas_forest[:, 1] < 0.60)
```

```
print(f"{{(y_train_5[idx_50_to_60]).sum() / idx_50_to_60.sum():.1%}}")
```

94.0%

```
y_scores_forest = y_probab_forest[:, 1]  
precisions_forest, recalls_forest, thresholds_forest = precision_recall_curve(  
    y_train_5, y_scores_forest)
```

```
plt.figure(figsize=(6, 5)) # extra code - not needed, just formatting
```

```
plt.plot(recalls_forest, precisions_forest, "b-", linewidth=2,  
         label="Random Forest")
```

```
plt.plot(recalls, precisions, "--", linewidth=2, label="SGD")
```

```
# extra code - just beautifies and saves Figure 3-8
```

```
plt.xlabel("Recall")
```

```
plt.ylabel("Precision")
```

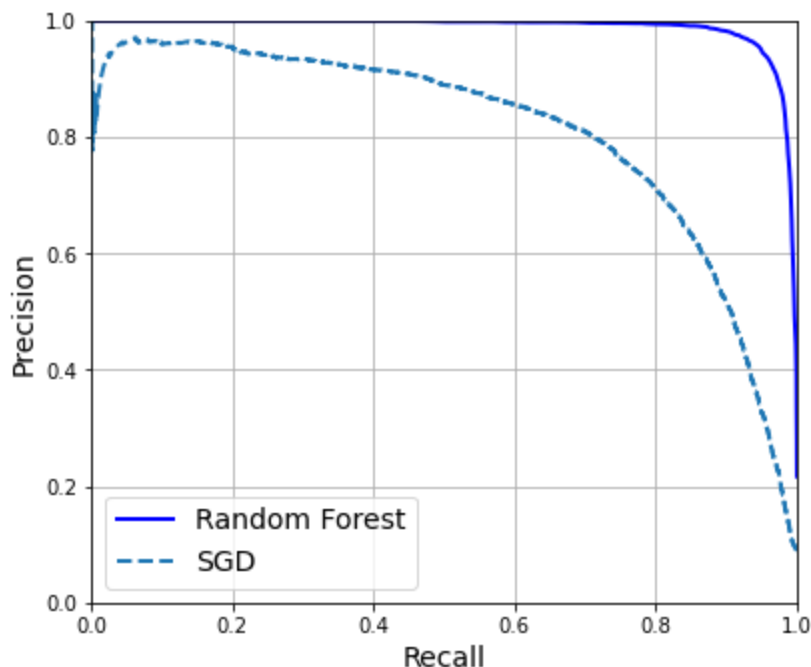
```
plt.axis([0, 1, 0, 1])
```

```
plt.grid()
```

```
plt.legend(loc="lower left")
```

```
save_fig("pr_curve_comparison_plot")
```

```
plt.show()
```



We could use `cross_val_predict(forest_clf, X_train, y_train_5, cv=3)` to compute `y_train_pred_forest`, but since we already have the estimated probabilities, we can just use the default threshold of 50% probability to get the same predictions much faster:

```
y_train_pred_forest = y_probas_forest[:, 1] >= 0.5 # positive proba ≥ 50%  
f1_score(y_train_5, y_train_pred_forest)
```

```
0.9274509803921569
```

```
roc_auc_score(y_train_5, y_scores_forest)
```

```
0.9983436731328145
```

```
precision_score(y_train_5, y_train_pred_forest)
```

```
0.9897468089558485
```

```
recall_score(y_train_5, y_train_pred_forest)
```

```
0.8725327430363402
```

```
# Phân loại đa lớp (Multiclass Classification)
```

SVMs do not scale well to large datasets, so let's only train on the first 2,000 instances, or else this section will take a very long time to run:

```
from sklearn.svm import SVC
```

```
svm_clf = SVC(random_state=42)  
svm_clf.fit(X_train[:2000], y_train[:2000]) # y_train, not y_train_5
```

```
SVC(random_state=42)
```

```
svm_clf.predict([some_digit])
```

```
array(['5'], dtype=object)
```

```
some_digit_scores = svm_clf.decision_function([some_digit])  
some_digit_scores.round(2)
```

```
array([[ 3.79,  0.73,  6.06,  8.3 , -0.29,  9.3 ,  1.75,  2.77,  7.21,  
        4.82]])
```

```
class_id = some_digit_scores.argmax()  
class_id
```

```
5
```

```
svm_clf.classes_
```

```
array(['0', '1', '2', '3', '4', '5', '6', '7', '8', '9'], dtype=object)
```

```
svm_clf.classes_[class_id]
```

```
'5'
```

If you want `decision_function()` to return all 45 scores, you can set the `decision_function_shape` hyperparameter to `"ovo"`. The default value is `"ovr"`, but don't let this confuse you: `SVC` always uses OvO for training. This hyperparameter only affects whether or not the 45 scores get aggregated or not:

```
# extra code - shows how to get all 45 OvO scores if needed
svm_clf.decision_function_shape = "ovo"
some_digit_scores_ovo = svm_clf.decision_function([some_digit])
some_digit_scores_ovo.round(2)
```

```
array([[ 0.11, -0.21, -0.97,  0.51, -1.01,  0.19,  0.09, -0.31, -0.04,
        -0.45, -1.28,  0.25, -1.01, -0.13, -0.32, -0.9 , -0.36, -0.93,
         0.79, -1.  ,  0.45,  0.24, -0.24,  0.25,  1.54, -0.77,  1.11,
         1.13,  1.04,  1.2 , -1.42, -0.53, -0.45, -0.99, -0.95,  1.21,
         1.  ,  1.  ,  1.08, -0.02, -0.67, -0.14, -0.3 , -0.13,  0.25]])
```

```
from sklearn.multiclass import OneVsRestClassifier
```

```
ovr_clf = OneVsRestClassifier(SVC(random_state=42))
ovr_clf.fit(X_train[:2000], y_train[:2000])
```

```
OneVsRestClassifier(estimator=SVC(random_state=42))
```

```
ovr_clf.predict([some_digit])
```

```
array(['5'], dtype='<U1')
```

```
len(ovr_clf.estimators_)
```

```
10
```

```
sgd_clf = SGDClassifier(random_state=42)
sgd_clf.fit(X_train, y_train)
sgd_clf.predict([some_digit])
```

```
array(['3'], dtype='<U1')
```

```
sgd_clf.decision_function([some_digit]).round()
```

```
array([[ -31893.,  -34420.,  -9531.,   1824.,  -22320.,  -1386.,  -26189.,
        -16148.,  -4604.,  -12051.]])
```

Warning: the following two cells may take a few minutes each to run:

```
cross_val_score(sgd_clf, X_train, y_train, cv=3, scoring="accuracy")
```

```
array([0.87365, 0.85835, 0.8689 ])
```

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train.astype("float64"))
cross_val_score(sgd_clf, X_train_scaled, y_train, cv=3, scoring="accuracy")
```

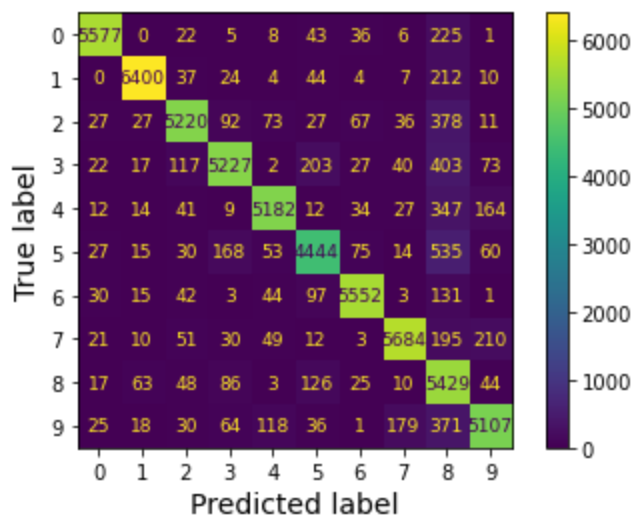
```
array([0.8983, 0.891 , 0.9018])
```

```
# Phân tích lỗi (Error Analysis)
```

Warning: the following cell will take a few minutes to run:

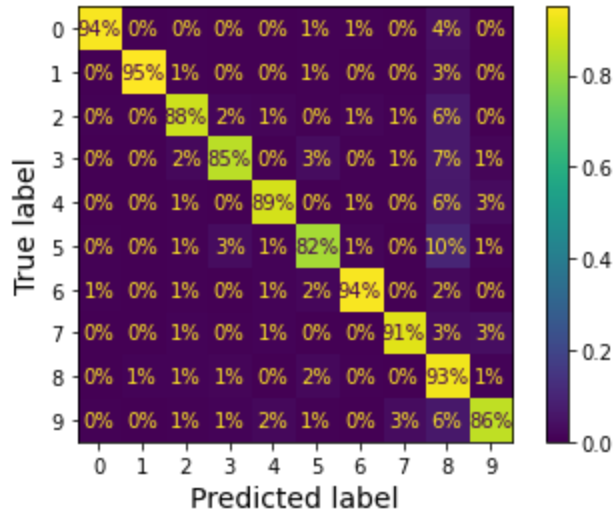
```
from sklearn.metrics import ConfusionMatrixDisplay
```

```
y_train_pred = cross_val_predict(sgd_clf, X_train_scaled, y_train, cv=3)
plt.rc('font', size=9) # extra code - make the text smaller
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred)
plt.show()
```



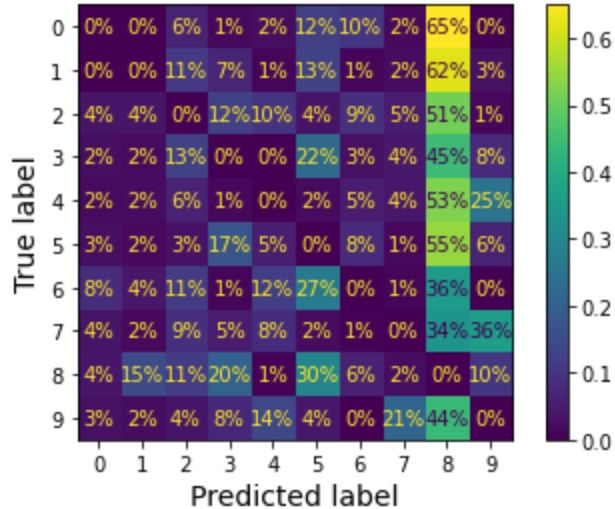
```
plt.rc('font', size=10) # extra code
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred,
```

```
plt.show()
normalize="true", values_format=".0%")
```



```
sample_weight = (y_train_pred != y_train)
plt.rc('font', size=10) # extra code
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred,
                                       sample_weight=sample_weight,
                                       normalize="true", values_format=".0%")

plt.show()
```



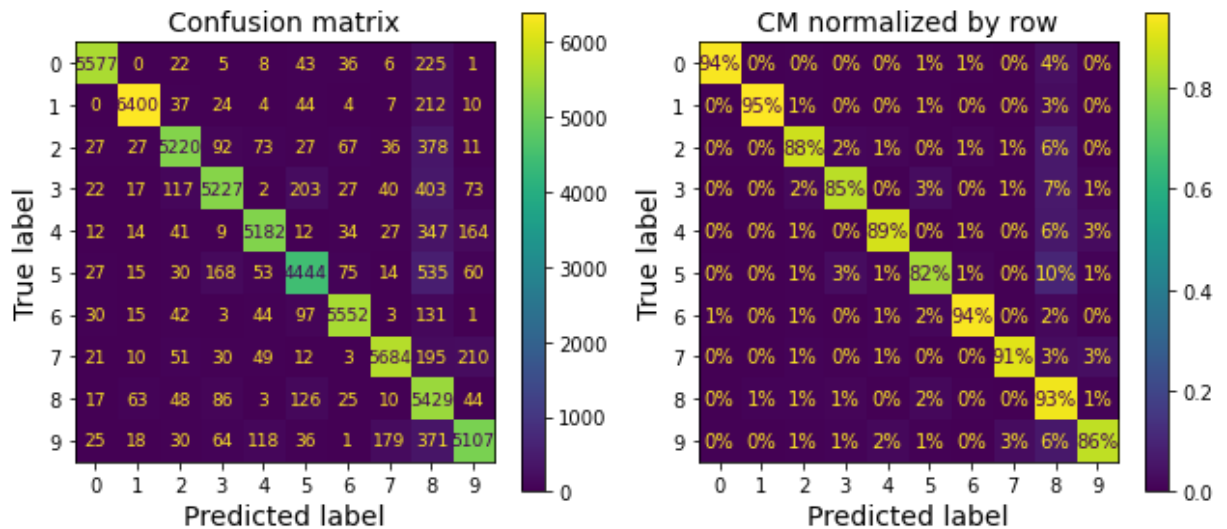
Let's put all plots in a couple of figures for the book:

```
# extra code - this cell generates and saves Figure 3-9
fig, axs = plt.subplots(nrows=1, ncols=2, figsize=(9, 4))
plt.rc('font', size=9)
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred, ax=axs[0])
axs[0].set_title("Confusion matrix")
plt.rc('font', size=10)
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred, ax=axs[1],
```

```

normalize="true", values_format=".0%")
axs[1].set_title("CM normalized by row")
save_fig("confusion_matrix_plot_1")
plt.show()

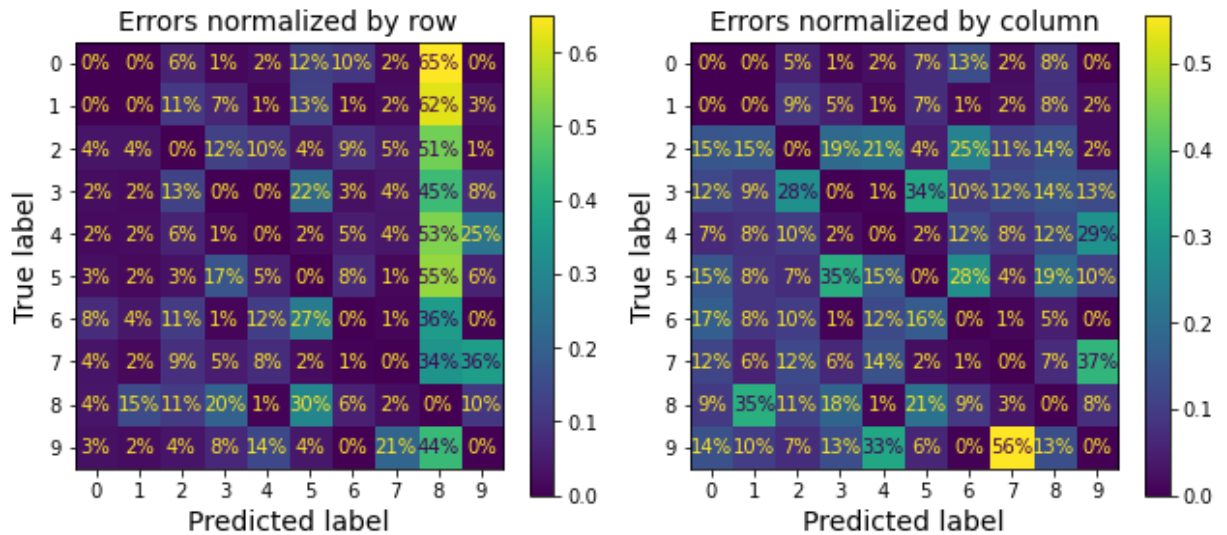
```



```

# extra code - this cell generates and saves Figure 3-10
fig, axs = plt.subplots(nrows=1, ncols=2, figsize=(9, 4))
plt.rc('font', size=10)
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred, ax=axs[0],
                                       sample_weight=sample_weight,
                                       normalize="true", values_format=".0%")
axs[0].set_title("Errors normalized by row")
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred, ax=axs[1],
                                       sample_weight=sample_weight,
                                       normalize="pred", values_format=".0%")
axs[1].set_title("Errors normalized by column")
save_fig("confusion_matrix_plot_2")
plt.show()
plt.rc('font', size=14) # make fonts great again

```



```

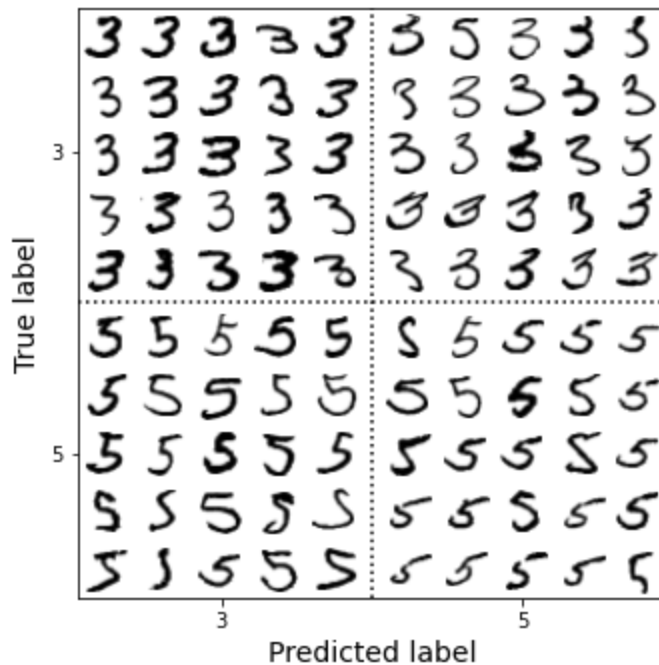
cl_a, cl_b = '3', '5'
X_aa = X_train[(y_train == cl_a) & (y_train_pred == cl_a)]
X_ab = X_train[(y_train == cl_a) & (y_train_pred == cl_b)]
X_ba = X_train[(y_train == cl_b) & (y_train_pred == cl_a)]
X_bb = X_train[(y_train == cl_b) & (y_train_pred == cl_b)]

```

```

# extra code - this cell generates and saves Figure 3-11
size = 5
pad = 0.2
plt.figure(figsize=(size, size))
for images, (label_col, label_row) in [(X_ba, (0, 0)), (X_bb, (1, 0)),
                                         (X_aa, (0, 1)), (X_ab, (1, 1))]:
    for idx, image_data in enumerate(images[:size*size]):
        x = idx % size + label_col * (size + pad)
        y = idx // size + label_row * (size + pad)
        plt.imshow(image_data.reshape(28, 28), cmap="binary",
                    extent=(x, x + 1, y, y + 1))
plt.xticks([size / 2, size + pad + size / 2], [str(cl_a), str(cl_b)])
plt.yticks([size / 2, size + pad + size / 2], [str(cl_b), str(cl_a)])
plt.plot([size + pad / 2, size + pad / 2], [0, 2 * size + pad], "k:")
plt.plot([0, 2 * size + pad], [size + pad / 2, size + pad / 2], "k:")
plt.axis([0, 2 * size + pad, 0, 2 * size + pad])
plt.xlabel("Predicted label")
plt.ylabel("True label")
save_fig("error_analysis_digits_plot")
plt.show()

```



Note: there are several other ways you could code a plot like this one, but it's a bit hard to get the axis labels right:

- using [nested GridSpecs](#)
- merging all the digits in each block into a single image (then using 2x2 subplots).
For example:

```
X_aa[:25].reshape(5, 5, 28, 28).transpose(0, 2, 1, 3).reshape(5 * 28, 5 * 28)
```

- using [subfigures](#) (since Matplotlib 3.4)

```
# Phân loại đa nhãn (Multilabel Classification)
```

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier

y_train_large = (y_train >= '7')
y_train_odd = (y_train.astype('int8') % 2 == 1)
y_multilabel = np.c_[y_train_large, y_train_odd]

knn_clf = KNeighborsClassifier()
knn_clf.fit(X_train, y_multilabel)

KNeighborsClassifier()
```

```
knn_clf.predict([some_digit])
```

```
array([[False,  True]])
```

Warning: the following cell may take a few minutes to run:

```
y_train_knn_pred = cross_val_predict(knn_clf, X_train, y_multilabel, cv=3)
f1_score(y_multilabel, y_train_knn_pred, average="macro")
```

```
0.976410265560605
```

```
# extra code - shows that we get a negligible performance improvement when we
#               set average="weighted" because the classes are already pretty
#               well balanced.
f1_score(y_multilabel, y_train_knn_pred, average="weighted")
```

```
0.9778357403921755
```

```
from sklearn.multioutput import ClassifierChain
```

```
chain_clf = ClassifierChain(SVC(), cv=3, random_state=42)
chain_clf.fit(X_train[:2000], y_multilabel[:2000])
```

```
ClassifierChain(base_estimator=SVC(), cv=3, random_state=42)
```

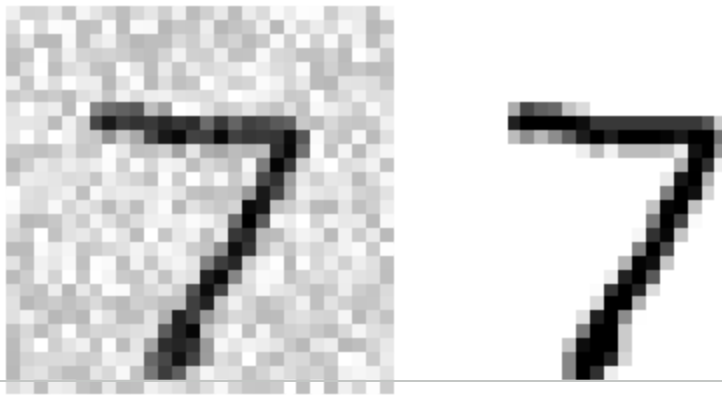
```
chain_clf.predict([some_digit])
```

```
array([[0.,  1.]])
```

Phân loại đa đầu ra (Multioutput Classification)

```
np.random.seed(42) # to make this code example reproducible
noise = np.random.randint(0, 100, (len(X_train), 784))
X_train_mod = X_train + noise
noise = np.random.randint(0, 100, (len(X_test), 784))
X_test_mod = X_test + noise
y_train_mod = X_train
y_test_mod = X_test
```

```
# extra code - this cell generates and saves Figure 3-12
plt.subplot(121); plot_digit(X_test_mod[0])
plt.subplot(122); plot_digit(y_test_mod[0])
save_fig("noisy_digit_example_plot")
plt.show()
```



```
knn_clf = KNeighborsClassifier()  
knn_clf.fit(X_train_mod, y_train_mod)  
clean_digit = knn_clf.predict([X_test_mod[0]])  
plot_digit(clean_digit)  
save_fig("cleaned_digit_example_plot") # extra code - saves Figure 3-13  
plt.show()
```

